



AI Mode

All

Shopping

Images

Videos

More



Sign in



urequests "Unsupported Transfer-Encoding: chunked"



The `urequests` module in MicroPython does not support chunked transfer encoding. When a server responds with `Transfer-Encoding: chunked`, `urequests` explicitly raises a `ValueError: Unsupported Transfer-Encoding: chunked`. [PyPI +2](#)

Why the Error Happens

- **Design limitation:** MicroPython's standard `urequests` library is intentionally lightweight and lacks the complex state machine needed to parse chunked data streams. [PyPI](#)
- **Server behavior:** The web server or API you are calling is choosing to stream the response using chunks rather than sending a fixed `Content-Length` header. [Stack Overflow +2](#)

How to Fix It

- **Force HTTP/1.0:** If you control the server, configure it to use HTTP/1.0 or disable chunked responses, which forces it to supply a standard `Content-Length` header. [Stack Overflow +2](#)
- **Change the endpoint/API:** If using a third-party API, check if you can pass parameters or headers that opt out of compression or streaming.
- **Switch libraries:** Migrate from `urequests` to `adafruit_requests` or an asynchronous alternative like `uaiohttpclient` if your specific target hardware and MicroPython build support them. [PyPI +1](#)

Ask anything



AI Mode

All

Shopping

Images

Videos

More ›



Sign in



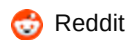
micropython-urequests - PyPI

It doesn't support automatic redirects or chunked transfer encoding. urequests implements a subset of API of the popular 3rd-party...



How to install packages with MIP : r/raspberrypipico - Reddit

File "requests/__init__.py", line 205, in get. File "requests/__init__.py", line 165, in request. ValueError: Unsupported Transfer...



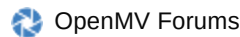
Reddit



ESP32 as a co-processor(slave) - OpenMV Forums

`adafruit\_requests`

===== ...

[Show all](#)

esp32 nodemcu

Since you are using an ESP32 NodeMCU, you have plenty of RAM and flash storage to install alternative libraries or a patched version of `urequests` that adds chunked streaming support. [↗](#)

Solution 1: Use an Upgraded `urequests` Script

The easiest fix is to replace the built-in `urequests` with a modified, drop-in replacement script that natively decodes chunked transfer encoding.



AI Mode

All

Shopping

Images

Videos

More ·



Sign in

```
import usocket

class Response:
    def __init__(self, f):
        self.raw = f
        self.encoding = "utf-8"
        self._cached = None

    def close(self):
        if self.raw:
            self.raw.close()
            self.raw = None
            self._cached = None

    @property
    def content(self):
        if self._cached is None:
            try:
                self._cached = self.raw.read()
            finally:
                self.raw.close()
                self.raw = None
        return self._cached

    @property
    def text(self):
        return str(self.content, self.encoding)

    def json(self):
        import ujson
        return ujson.loads(self.content)

def request(method, url, data=None, json=None, headers={}, stream=
```



AI Mode

All

Shopping

Images

Videos

More ›



Sign in

```
        port = 443
    else:
        raise ValueError("Unsupported scheme")

    if ":" in host:
        host, port = host.split(":", 1)
        port = int(port)

    ai = usocket.getaddrinfo(host, port, 0, usocket.SOCK_STREAM)
    ai = ai[0]
    s = usocket.socket(ai[0], ai[1], ai[2])

    if timeout is not None:
        s.settimeout(timeout)

    try:
        s.connect(ai[4])
        if scheme == "https:":
            s = ssl.wrap_socket(s, server_hostname=host)

        # Send HTTP request headers
        s.write(b"%s %s HTTP/1.1\r\n" % (method.encode(), path.encode()))
        if "Host" not in headers:
            s.write(b"Host: %s\r\n" % host.encode())

        for k, v in headers.items():
            s.write(b"%s: %s\r\n" % (k.encode(), v.encode()))

        if json is not None:
            import ujson
            data = ujson.dumps(json)
            s.write(b"Content-Type: application/json\r\n")
```



AI Mode

All

Shopping

Images

Videos

More ·



Sign in

```
l = f.readline()
l = l.split(None, 2)
status = int(l[1])

chunked = False
while True:
    l = f.readline()
    if not l or l == b"\r\n":
        break
    if l.lower().startswith(b"transfer-encoding:"):
        if b"chunked" in l.lower():
            chunked = True

# Wrap stream if it's chunked to decode it transparently
if chunked:
    f = ChunkedReader(f)

resp = Response(f)
resp.status_code = status
return resp
except Exception as e:
    s.close()
    raise e

class ChunkedReader:
    def __init__(self, f):
        self.f = f
        self.chunk_left = 0

    def read(self, sz=-1):
        if sz != -1:
            raise NotImplementedError("Arbitrary chunk sizing is not
            out = bytearray()
```



AI Mode

All

Shopping

Images

Videos

More ·



Sign in

```
        out.extend(data)
        self.chunk_left -= len(data)
    return bytes(out)

    def close(self):
        self.f.close()

    def get(url, **kw): return request("GET", url, **kw)
    def post(url, **kw): return request("POST", url, **kw)
```

Use code with caution.



Solution 2: Change Headers to Force No-Chunking

Sometimes you can ask the server politely to give you a raw standard response instead of chunks. Add the following header fields to your existing

`urequests.get()` call:

python

```
headers = {
    "Connection": "close",
    "Accept-Encoding": "identity"
}
response = urequests.get("http://your-api.url", headers=headers)
```

Use code with caution.



- **Accept-Encoding: identity** directly asks the server not to use chunked



AI Mode

All

Shopping

Images

Videos

More ·



Sign in

```
import mip  
mip.install("requests")
```

Use code with caution.



Then use `import requests` in your python file instead of `urequests`.

If you can tell me **which API/URL you are calling** or **provide a copy of your request script**, I can show you exactly how to patch it.

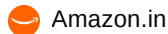


3 sites



BITSKY INDIA ESP32 WROOM-32 Development Board CP2102 USB Driver 2.4GHz Dual Core WiFi + Bluetooth (30 PINS)

Top highlights ESP32 (ESP32S) Chip: The heart of the NodeMCU board, featuring the ESP32 (ESP32S) dual-core processor. Flash Me...



Amazon.in



ESP-WROOM-32 38 Pins WiFi Bluetooth Module

NodeMCU is famous for the ESP8266E module with the LUA programming language. Now, this is a more powerful NodeMCU with ESP32 on it...

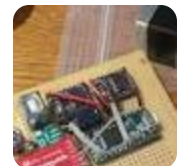


AM Electronics



ESP8266 and ESP32 as the brain for OnStep?

Dec 16, 2018 — They can be packaged with a microUSB and breakout pins for PWM, I2C, ...etc. Examples are NodeMCU, Wemos D1, Feather Huzzah, ...et...



AI Mode

All

Shopping

Images

Videos

More





Sign in



